

Distributed Computing – Question Bank with Answers

Part A – 2 Marks Questions

1. Define a Distributed System. (Unit-I)

Answer:

A distributed system is a collection of independent computers that communicate through a network and cooperate to perform a common task.

Key points:

1. It consists of multiple autonomous computers.
2. The computers communicate through a network.
3. Resources and processing are shared among the systems.
4. To users, the distributed system appears as a single coherent system.

Example: Cloud computing platforms, distributed databases, and online banking systems.

2. State the differences between synchronous and asynchronous communication along with the buffering strategy adopted in both. (Unit-I)

Answer:

Synchronous Communication	Asynchronous Communication
Sender and receiver synchronize during communication.	Sender and receiver need not synchronize.
Sender may wait until the receiver is ready.	Sender can continue execution after sending.
Usually requires coordination between processes.	Uses message buffering to store messages.
Buffering requirement is generally low or bounded.	Requires message queues/buffers to hold messages.

Buffering strategies:

1. **Zero buffering:** Sender waits until the receiver is ready.
2. **Bounded buffering:** A finite number of messages can be stored.
3. **Unbounded buffering:** Messages are stored without a fixed buffer limit.
4. Asynchronous communication commonly uses **message queues**.

3. Distinguish between Message Passing Systems and Shared Memory Systems. (Unit-I)

Answer:

Message Passing

Shared Memory

Message Passing

Processes communicate by sending and receiving messages.

No common address space is required.

Suitable for distributed systems.

Communication is explicit.

Shared Memory

Processes communicate by reading/writing shared data.

A common memory space is required.

Commonly used in multiprocessor systems.

Communication occurs through shared variables/data structures.

Example:

Message passing → `send()` and `receive()`

Shared memory → common buffer accessed by multiple processes.

4. Define Concurrency and Granularity. (Unit-I)

Answer:

Concurrency is the ability of multiple processes to execute or make progress during overlapping periods of time.

Granularity refers to the size of the computational tasks into which a program is divided.

Key points:

1. Concurrent processes can execute independently.
2. Concurrency improves resource utilization.
3. Fine-grained tasks contain small amounts of computation.
4. Coarse-grained tasks contain relatively large amounts of computation.

Example: Dividing a large matrix operation into several independent tasks provides concurrency.

5. What are the motivations for Distributed Systems? (Unit-I)

Answer:

The major motivations are:

1. **Resource sharing** – hardware, software, data and services can be shared.
2. **Performance improvement** – tasks can be executed concurrently.
3. **Reliability** – failure of one system need not stop the entire service.
4. **Scalability** – additional computers can be added as requirements increase.

Other motivations include geographical distribution, communication, cost effectiveness and availability.

6. What are the Message Ordering Paradigms? (Unit-II)

Answer:

Message ordering specifies the order in which messages are delivered to processes.

Four important ordering paradigms are:

1. **FIFO ordering** – messages from the same sender are delivered in their sending order.
2. **Causal ordering** – causally related messages are delivered in causal order.
3. **Total ordering** – all processes deliver messages in the same order.
4. **Causal-total ordering** – preserves causal relationships while maintaining a common total order.

7. State the property for causal delivery of messages. (Unit-II)

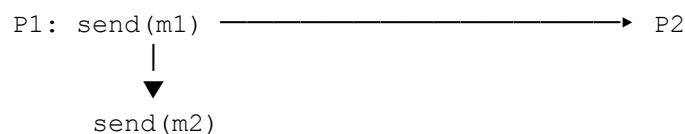
Answer:

For causal delivery, if message m_1 causally precedes m_2 , then every process that delivers both messages must deliver m_1 before m_2 .

Properties:

1. Causal relationship must be preserved.
2. If $m_1 \rightarrow m_2$, then $\text{deliver}(m_1)$ occurs before $\text{deliver}(m_2)$.
3. Concurrent messages do not have a mandatory ordering.
4. Causal ordering can be represented using **Lamport clocks or vector clocks**.

Example:



Therefore: $m_1 \rightarrow m_2$
Delivery: m_1 before m_2

8. How do you define Scalar Time and Vector Time? (Unit-II)

Answer:

Scalar time: A single integer value is assigned to events to represent their logical ordering.

Vector time: A vector containing logical clock values for all processes is used to represent causal relationships.

Key points:

1. Scalar clocks use one value per event.
2. Lamport's logical clock is an example of scalar time.
3. Vector clocks use multiple values.
4. Vector clocks provide more information about causal relationships.

Example:

Scalar time: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Vector time:

P1 = [2, 0, 1]

P2 = [2, 3, 1]

P3 = [2, 3, 4]

9. Distinguish between Causal Order and Total Order in Group Communication. (Unit-II)

Answer:

Causal Order	Total Order
Preserves causal relationships between messages.	Provides the same delivery order at all processes.
Concurrent messages may be delivered in different orders.	Even concurrent messages have the same order.
Uses causal information such as vector clocks.	Uses a global ordering mechanism.
Weaker than total ordering.	Stronger ordering guarantee.

Example:

Causal:

$m1 \rightarrow m2$

Therefore m1 must be delivered before m2.

Total:

P1: $m1 \rightarrow m2 \rightarrow m3$

P2: $m1 \rightarrow m2 \rightarrow m3$

P3: $m1 \rightarrow m2 \rightarrow m3$

10. What are the limitations of Logical Clock? (Unit-II)

Answer:

The limitations of logical clocks include:

1. They provide ordering of events, but not actual physical time.
2. Lamport clocks cannot determine whether two events are concurrent.

3. Different events may have the same logical ordering relationship.
4. They cannot completely represent causal relationships among all distributed events.

Example:

If:

$$L(a) < L(b)$$

it does not necessarily mean:

$$a \rightarrow b$$

The events may actually be concurrent.

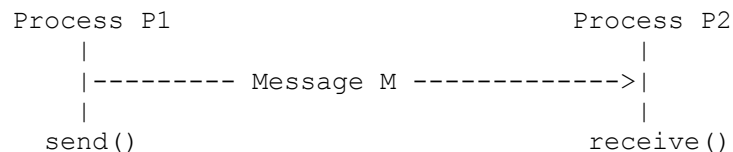
11. What do you mean by Message Passing? (Unit-I)

Answer:

Message passing is a communication mechanism in which processes exchange information by explicitly sending and receiving messages.

1. A sender process sends a message to a receiver process.
2. The receiver obtains the message using a receive operation.
3. Processes need not share a common memory.
4. It is widely used for communication between processes in distributed systems.

Diagram:



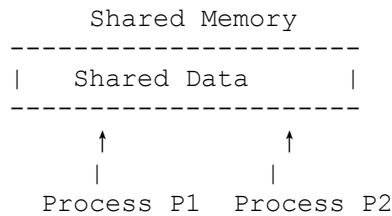
12. List the role of Shared Memory Systems. (Unit-I)

Answer:

Shared memory systems provide a common memory area that can be accessed by multiple processes.

1. It allows processes to **share data** efficiently.
2. It provides a mechanism for **inter-process communication**.
3. Processes can communicate through shared variables and data structures.
4. Synchronization mechanisms such as **locks, semaphores and mutual exclusion** are required.

Diagram:

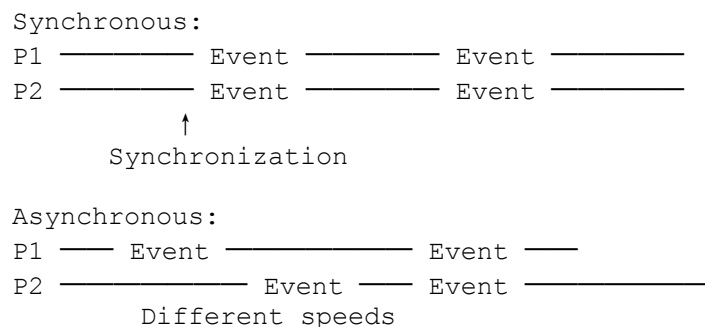


13. What is the difference between Asynchronous and Synchronous Execution? (Unit-I)

Answer:

1. **Synchronous execution** requires processes to proceed according to a coordinated timing or synchronization model.
2. **Asynchronous execution** does not assume a common timing or execution speed.
3. In synchronous execution, communication and events may have known timing constraints.
4. In asynchronous execution, processes can execute at different speeds and message delays are unpredictable.

Diagram:



14. Enlist the Design Issues and Challenges of Distributed Systems. (Unit-I)

Answer:

The major design issues are:

1. **Heterogeneity** – different hardware, operating systems and networks.
2. **Openness** – interoperability and extensibility.
3. **Security** – protecting distributed resources and communication.
4. **Scalability** – maintaining performance as the system grows.

Other issues include transparency, fault tolerance, synchronization, resource management and concurrency.

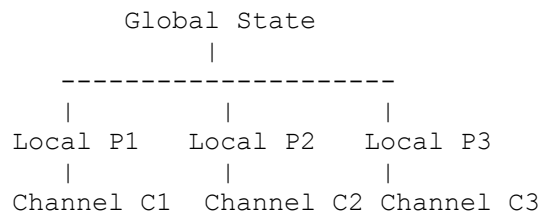
15. What is meant by Global State in a Distributed System? (Unit-I)

Answer:

The **global state** of a distributed system is the combination of the states of all processes and the states of communication channels at a particular logical instant.

1. It includes the local state of every process.
2. It includes the state of communication channels.
3. There is no single global clock in a distributed system.
4. Global state is useful for **deadlock detection, termination detection and checkpointing**.

Diagram:



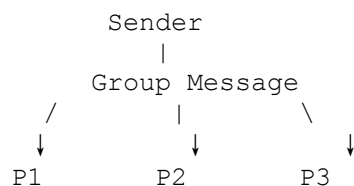
16. What is meant by Group Communication? (Unit-II)

Answer:

Group communication is a communication mechanism in which a message is sent from one process to a group of processes rather than to only one process.

1. A collection of processes forms a communication group.
2. A sender can transmit a message to multiple group members.
3. It supports operations such as multicast and broadcast.
4. It is useful in replicated services, fault-tolerant systems and distributed applications.

Diagram:



17. What are the Forms of Message Ordering Paradigms? (Unit-II)

Answer:

The major forms of message ordering are:

1. **FIFO ordering** – messages from the same sender are delivered in sending order.
2. **Causal ordering** – causally related messages are delivered in causal order.
3. **Total ordering** – all processes deliver messages in the same order.
4. **Causal-total ordering** – combines causal ordering with a common total order.

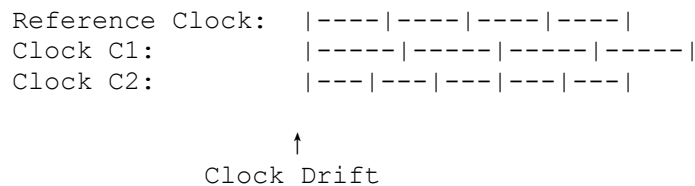
18. What is Clock Drift Rate? (Unit-II)

Answer:

Clock drift rate is the rate at which a physical clock deviates from a reference clock due to differences in clock hardware.

1. Every physical clock may run slightly faster or slower.
2. This difference accumulates over time.
3. Clock drift causes different computers to show different times.
4. Clock synchronization mechanisms such as **NTP** are used to reduce clock differences.

Diagram:



19. State the Issues in Clocks. (Unit-II)

Answer:

Important issues associated with physical clocks are:

1. **Clock drift** – clocks run at slightly different rates.
2. **Clock skew** – difference between the readings of two clocks.
3. **Clock synchronization** – maintaining clocks close to a common reference.
4. **Network delay** – message transmission time affects synchronization accuracy.

20. Define Causal Precedence Relation. (Unit-II)

Answer:

The **causal precedence relation**, usually represented by $\rightarrow$, indicates that one event can causally influence another event.

1. If two events occur in the same process in sequence, the earlier event precedes the later event.
2. A send event causally precedes its corresponding receive event.

3. The relation is transitive.
4. If $a \rightarrow b$, then a must occur before b in the causal ordering.

Example:

P1: a — send(m1) —————
 \
 \
P2: receive(m1) — b

Therefore: send(m1) $\rightarrow$ receive(m1)

21. What is meant by transparency in a distributed system? (Unit -I)

Answer:

Transparency means hiding the distributed nature of the system from users and applications.

1. It makes distributed resources appear as a single system.
2. **Location transparency** hides the physical location of resources.
3. **Access transparency** hides differences in accessing local and remote resources.
4. **Failure transparency** hides failures and recovery from users.

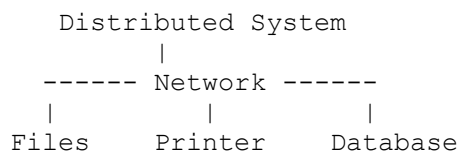
22. What is resource sharing in distributed systems? (Unit-I)

Answer:

Resource sharing is the ability to access and use resources located on different computers.

1. Resources may include hardware, software and data.
2. Files and databases can be shared among multiple users.
3. Printers and storage devices can be shared through a network.
4. Resource sharing improves utilization and reduces duplication.

Example:



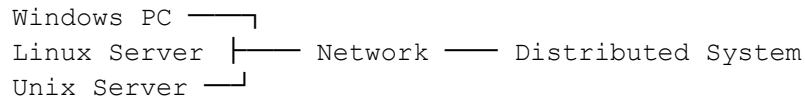
23. What is meant by heterogeneity in distributed systems? (Unit-I)

Answer:

Heterogeneity refers to the presence of different types of components in a distributed system.

1. Different hardware platforms may be connected.
2. Different operating systems may be used.
3. Different programming languages may be supported.
4. Different networks and communication protocols may coexist.

Example:



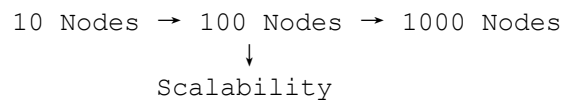
24. What is scalability in a distributed system? (Unit-I)

Answer:

Scalability is the ability of a distributed system to continue functioning effectively as its size or workload increases.

1. More users can be supported.
2. More computers or nodes can be added.
3. Data and service capacity can be increased.
4. The system should avoid significant performance degradation.

Example:



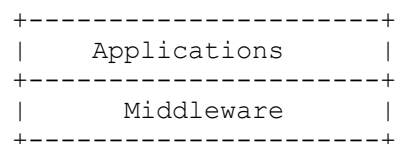
25. What is middleware in a distributed system? (Unit-I)

Answer:

Middleware is a software layer that provides common services between distributed applications and the underlying operating systems and networks.

1. It hides differences between heterogeneous systems.
2. It provides communication services.
3. It supports services such as naming and synchronization.
4. It simplifies distributed application development.

Diagram:



```
| Operating Systems      |  
+-----+  
| Hardware / Network    |  
+-----+
```

26. What is Lamport's logical clock? (Unit-II)

Answer:

Lamport's logical clock is a mechanism used to assign timestamps to events in a distributed system.

1. Each process maintains a logical counter.
2. The counter is incremented before an event.
3. A message carries the sender's timestamp.
4. The receiver updates its clock using the received timestamp.

Basic rule:

$$L(\text{receive}) = \max(L(\text{receive}), L(\text{message})) + 1$$

27. What is a vector clock? (Unit-II)

Answer:

A vector clock is a logical clock that uses a vector of values to represent causal relationships among distributed events.

1. Each process maintains a vector.
2. The vector contains one entry for each process.
3. It provides more causal information than a scalar clock.
4. It can identify whether two events are causally related or concurrent.

Example:

```
P1 → [2, 0, 0]  
P2 → [2, 3, 0]  
P3 → [2, 3, 4]
```

28. What is FIFO message ordering? (Unit-II)

Answer:

FIFO (First-In-First-Out) ordering ensures that messages sent by the same sender are delivered in the order in which they were sent.

1. Messages from one sender preserve their sending order.

2. If m_1 is sent before m_2 , m_1 is delivered before m_2 .
3. It does not necessarily order messages from different senders.
4. It is useful for maintaining sender-wise consistency.

Example:

P1: send(m_1) → send(m_2) → send(m_3)

P2: receive(m_1) → receive(m_2) → receive(m_3)

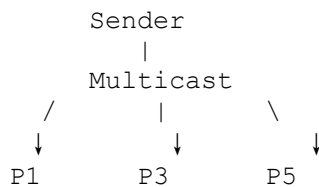
29. What is multicast communication? (Unit-II)

Answer:

Multicast is a communication mechanism in which a message is sent from one process to a selected group of processes.

1. The sender transmits one message to a group.
2. Only members of the specified group receive it.
3. It reduces unnecessary communication compared with separate unicast messages.
4. It is useful in replicated services and collaborative applications.

Diagram:



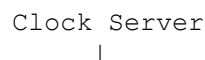
30. What is clock synchronization? (Unit-II)

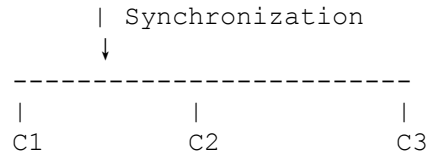
Answer:

Clock synchronization is the process of adjusting the physical clocks of distributed computers so that their time readings remain sufficiently close.

1. Each computer has its own physical clock.
2. Clock rates may differ because of clock drift.
3. Network delays affect synchronization.
4. Algorithms such as **Cristian's algorithm**, **Berkeley algorithm** and **NTP** are used for synchronization.

Diagram:





Part B – 10 Marks Questions

11(a). Explain the Design Issues and Challenges in the Design of Distributed Operating Systems. (Unit-I)

Answer

A Distributed Operating System (DOS) manages a collection of independent computers and provides users with the illusion of a single integrated computing system.

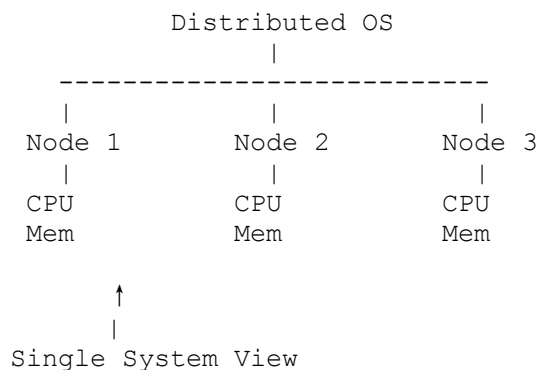
The major design issues and challenges are as follows.

1. Transparency

The system should hide the distributed nature of resources from users.

Types include:

- Location transparency
- Access transparency
- Migration transparency
- Replication transparency
- Concurrency transparency
- Failure transparency



2. Communication

Processes running on different machines must communicate efficiently.

Important mechanisms include:

- Message passing
- Remote procedure calls
- Remote method invocation
- Message-oriented middleware

3. Synchronization

Distributed processes must coordinate their activities without a common global clock.

Examples:

- Logical clocks
- Physical clock synchronization
- Mutual exclusion
- Distributed synchronization algorithms

4. Resource Management

The operating system must manage distributed resources such as:

- CPUs
- Memory
- Files
- Printers
- Storage
- Network resources

Efficient resource allocation is required to avoid underutilization.

5. Process Management

The system must support processes executing on different computers.

Important issues include:

- Process creation
- Process migration
- Process scheduling
- Load balancing
- Process coordination

6. Fault Tolerance

Failures are common in distributed systems because many independent components are involved.

The system should handle:

- Machine failures
- Communication failures
- Process failures
- Network partitioning

Techniques include replication, checkpointing and recovery.

7. Security

Distributed systems require protection of data and communication.

Major security mechanisms include:

- Authentication
- Authorization
- Encryption
- Access control
- Secure communication

8. Scalability

The system should continue to work efficiently when the number of users, processes or computers increases.

```
Small system
  ↓
10 nodes
  ↓
100 nodes
  ↓
1000 nodes
  ↓
Large distributed system
```

9. Heterogeneity

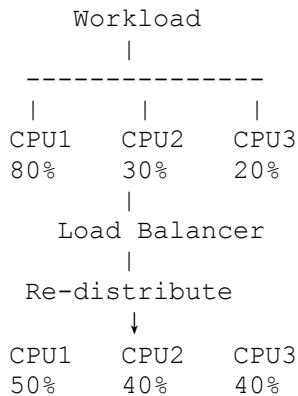
Distributed systems may contain different:

- Hardware platforms
- Operating systems
- Programming languages
- Network technologies
- Data representations

The DOS must provide interoperability between heterogeneous components.

10. Load Balancing

Work should be distributed among available processors to prevent some nodes from becoming overloaded.



Conclusion

The major challenge in distributed OS design is to provide transparency, efficient communication, synchronization, resource sharing, fault tolerance, security and scalability while hiding the complexity of the underlying distributed environment.

11(b). Analyze the Concepts of Heterogeneity, Openness, Security and Scalability Impact of Distributed Systems. (Unit-I)

Answer

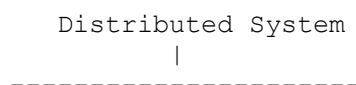
A distributed system consists of multiple independent components connected through a network. Heterogeneity, openness, security and scalability significantly influence its design.

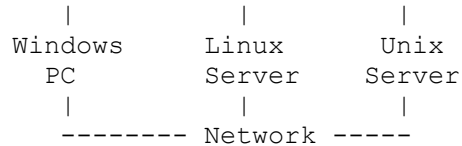
1. Heterogeneity

Heterogeneity means that different types of hardware, software and communication technologies coexist in the system.

Examples:

- Different processors
- Different operating systems
- Different programming languages
- Different networks
- Different data formats





Impact

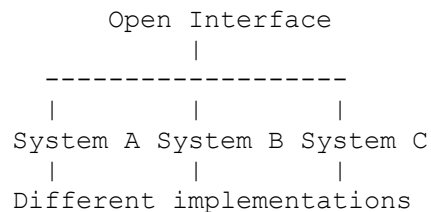
- Requires interoperability.
- Makes communication more complex.
- Requires common protocols and interfaces.
- Middleware is often used to hide differences.

2. Openness

An open distributed system supports the addition, modification and replacement of components using standard interfaces.

Characteristics

1. Standard interfaces.
2. Interoperability.
3. Portability.
4. Extensibility.



Impact

Openness makes it easier to:

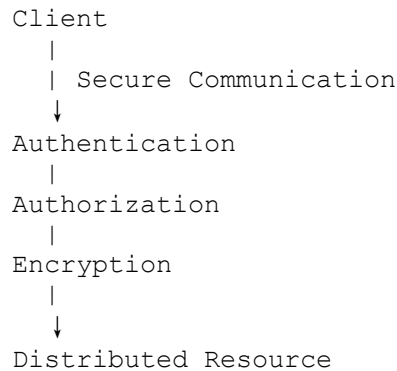
- Add new services.
- Replace components.
- Integrate different systems.
- Develop applications independently.

3. Security:

Security protects distributed resources and communication against unauthorized access and attacks.

Main security requirements

1. **Confidentiality** – prevents unauthorized disclosure.
2. **Integrity** – prevents unauthorized modification.
3. **Availability** – ensures resources remain accessible.
4. **Authentication** – verifies user or system identity.



Impact

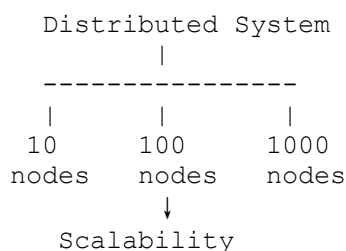
Security mechanisms may introduce additional processing and communication overhead, but they are essential for protecting distributed resources.

4. Scalability

Scalability is the ability of a distributed system to remain effective as the system size increases.

A system may scale in:

- Number of users
- Number of computers
- Geographic area
- Number of resources
- Data volume



Challenges

- Increased communication overhead.
- Resource management complexity.
- Performance degradation.
- Increased failure probability.

- Synchronization difficulties.

Conclusion

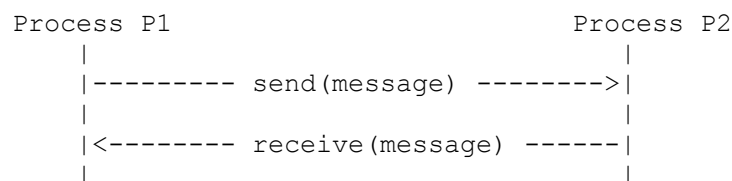
Heterogeneity requires interoperability, openness promotes extensibility, security protects resources and scalability enables the system to grow. All four must be considered carefully while designing distributed systems.

12(a). Explain Message Passing Systems and Discuss Message-Oriented Middleware and its Types and Functionality in Distributed Computing. (Unit-I)

Answer

Message passing is a communication mechanism in which processes exchange information by explicitly sending and receiving messages.

The basic operations are:



1. Message Passing System

The two fundamental operations are:

Send

A sender transmits a message to another process.

```
send(destination, message)
```

Receive

The receiver obtains the message.

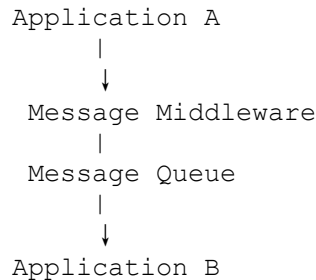
```
receive(source, message)
```

Characteristics

1. No shared memory is required.
2. Communication occurs through messages.
3. Processes can execute on different machines.
4. Suitable for distributed environments.

2. Message-Oriented Middleware

Message-Oriented Middleware (MOM) provides communication services between distributed applications using messages.



It separates the sender and receiver and can provide reliable message delivery.

3. Types of Message-Oriented Middleware

a) Point-to-Point

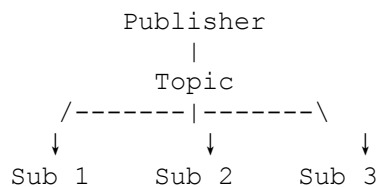
A message is sent to a specific destination or queue.

Producer → Queue → Consumer

One consumer normally receives a particular message.

b) Publish/Subscribe

A publisher sends messages to a topic, and multiple subscribers receive them.



4. Functionality of MOM

1. **Message queuing** – stores messages until they can be processed.
2. **Asynchronous communication** – sender need not wait for receiver.
3. **Message routing** – delivers messages to appropriate destinations.
4. **Reliability** – supports reliable message delivery.
5. **Persistence** – messages may be stored temporarily.
6. **Loose coupling** – sender and receiver do not need to operate simultaneously.
7. **Scalability** – supports communication among many distributed components.

Conclusion

Message passing provides the basic communication mechanism, while message-oriented middleware provides higher-level services such as queuing, routing, persistence and asynchronous communication.

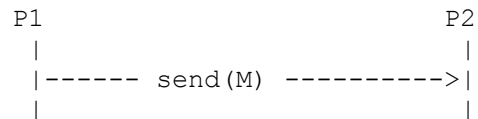
12(b). Discuss the Various Primitives for Distributed Communication. (Unit-I)

Answer

Communication primitives are basic operations used by distributed processes to exchange information.

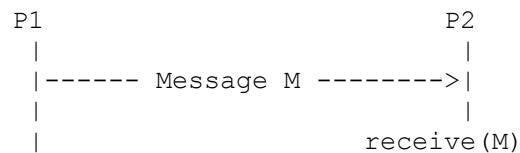
1. Send Primitive

The **send** operation transmits a message from one process to another.



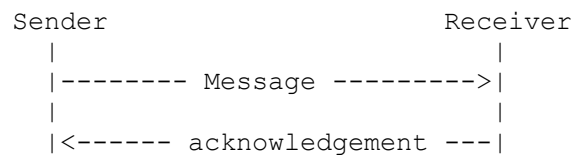
2. Receive Primitive

The **receive** operation obtains a message sent by another process.



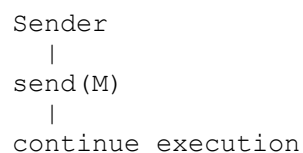
3. Blocking Send

The sender waits until the communication operation reaches the required synchronization point.



4. Non-blocking Send

The sender sends a message and continues execution without waiting for the receiver.



5. Blocking Receive

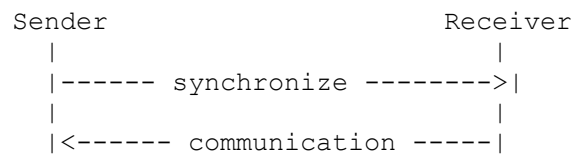
The receiver waits until a message becomes available.

6. Non-blocking Receive

The receiver checks for a message and continues if no message is available.

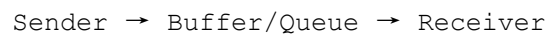
7. Synchronous Communication

Both sender and receiver participate in a synchronization event.



8. Asynchronous Communication

The sender and receiver operate independently using buffers or queues.



9. Buffered Communication

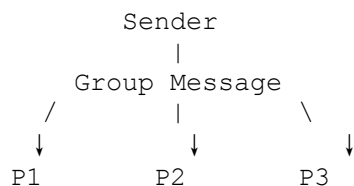
Messages are temporarily stored in buffers.

Types:

- Zero capacity
- Bounded capacity
- Unbounded capacity

10. Group Communication

A message can be transmitted to multiple processes.



Conclusion

Send, receive, blocking/non-blocking and synchronous/asynchronous primitives provide the foundation for process communication in distributed systems.

13(a). Explain How a Parallel System Differs from a Distributed System. (Unit-I)

Answer

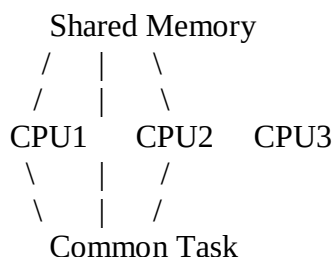
Parallel and distributed systems both use multiple computing resources, but they differ in architecture, communication and resource organization.

Comparison

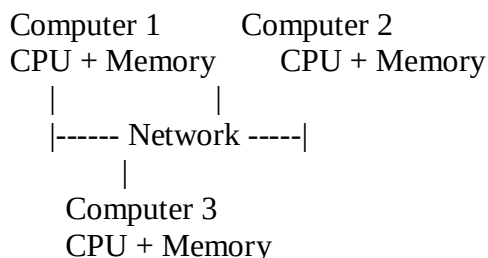
Parallel System	Distributed System
Multiple processors work together on a problem.	Multiple autonomous computers cooperate.
Processors are usually closely coupled.	Computers are loosely coupled.
Often uses shared memory.	Usually uses message passing/network communication.
Communication is relatively fast.	Communication involves a network.
Usually has centralized control or tightly coordinated control.	Control is generally distributed.
Components are often homogeneous.	Components may be heterogeneous.
Failure handling is comparatively simpler.	Failure handling is more complex.
Mainly focuses on improving computational performance.	Focuses on resource sharing, scalability, reliability and communication.

Diagram

Parallel System



Distributed System



Major Differences

1. Coupling

Parallel systems are generally tightly coupled, whereas distributed systems are loosely coupled.

2. Memory

Parallel systems may share memory, while distributed systems generally have separate local memories.

3. Communication

Parallel systems often use high-speed interconnection networks, whereas distributed systems use network communication.

4. Scalability

Distributed systems can be expanded by adding independent computers.

5. Fault Tolerance

Distributed systems must deal with independent failures of computers and communication links.

Conclusion:

A parallel system primarily aims at high-speed computation through simultaneous execution, whereas a distributed system aims at resource sharing, scalability, reliability and cooperation among autonomous computers.

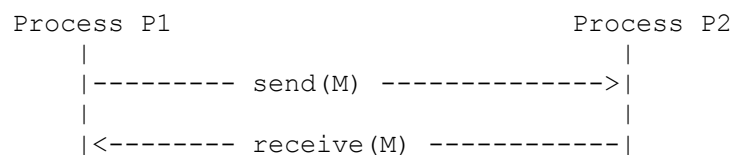
13(b). Illustrate the Difference Between Message Passing and Shared Memory Process Communication Models. (Unit-I)

Answer

Distributed processes can communicate mainly through message passing or shared memory.

1. Message Passing Model

Processes communicate by explicitly exchanging messages.

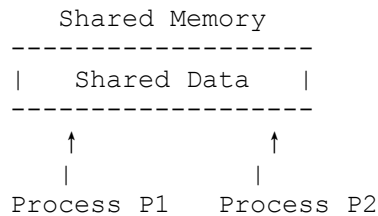


Features

1. Processes have separate address spaces.
2. Communication is explicit.
3. `send()` and `receive()` are fundamental operations.
4. Suitable for distributed systems.
5. Network communication is required when processes are on different machines.

2. Shared Memory Model

Processes communicate by accessing a common memory region.



Features

1. Processes access common data.
2. Communication occurs through shared variables.
3. Synchronization is necessary.
4. Mutual exclusion may be required.
5. Common in tightly coupled multiprocessor systems.

Comparison

Message Passing	Shared Memory
Explicit communication	Implicit communication through shared data
Uses send/receive	Uses read/write
Separate address spaces	Common address space
Suitable for distributed systems	Common in multiprocessor systems
Synchronization is associated with communication	Requires synchronization mechanisms for shared data

Conclusion

Message passing is appropriate when processes are separated by a network, while shared memory is efficient when processes can access a common memory space.

14(a). Explain How Physical Clocks are Synchronized in Distributed Systems. (Unit - II)

Answer

A physical clock measures real-world time. In a distributed system, each computer has its own physical clock, and these clocks may not show exactly the same time because of clock drift and differences in hardware.

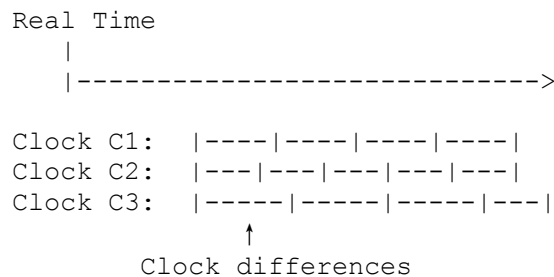
1. Need for Clock Synchronization

Synchronization is required for:

- Correct event ordering.
- Timestamping.
- Distributed transactions.
- File management.
- Security protocols.
- Coordinating distributed processes.

2. Clock Drift

Different hardware clocks run at slightly different rates.

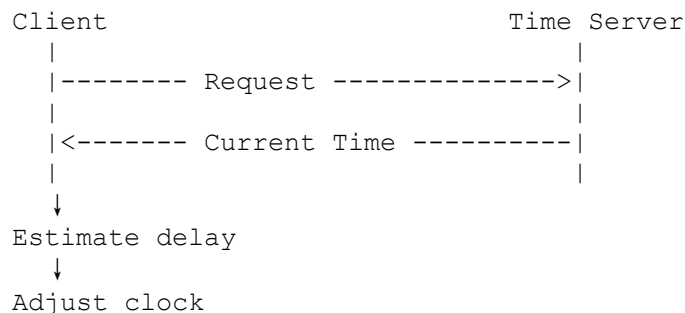


3. Cristian's Algorithm

Cristian's algorithm uses a time server.

Steps

1. Client sends a request to the time server.
2. Server returns its current time.
3. Client estimates network delay.
4. Client adjusts its clock.



Approximate adjustment:

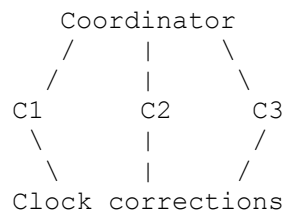
New time = Server time + estimated communication delay

4. Berkeley Algorithm

The Berkeley algorithm uses a master or coordinator to synchronize clocks.

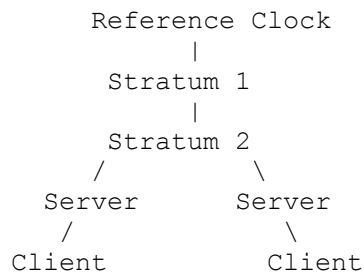
Steps

1. Coordinator polls all machines.
2. It obtains their clock values.
3. It calculates an average or suitable adjustment.
4. It sends corrections to the machines.
5. Each machine adjusts its clock.



5. Network Time Protocol (NTP)

NTP synchronizes clocks over computer networks using a hierarchy of time servers.



Conclusion

Physical clock synchronization is essential in distributed systems. Cristian's algorithm, Berkeley algorithm and NTP are important approaches used to reduce differences between physical clocks.

14(b). Summarize NTP for Synchronizing System of Physical Clocks in Distributed Systems. (Unit - II)

Answer

Network Time Protocol (NTP) is a protocol used to synchronize the clocks of computers over a network.

1. Purpose of NTP

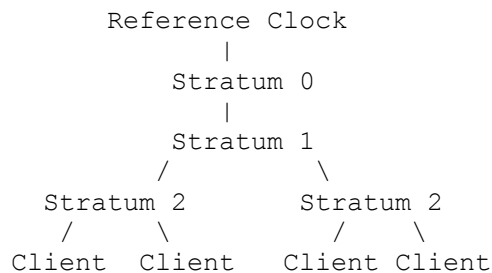
NTP provides a common time reference for distributed computers.

It is used for:

- Event timestamping.
- Log management.
- Distributed transactions.
- Network operations.
- Security mechanisms.

2. NTP Hierarchy

NTP uses a hierarchical structure called **strata**.



Stratum 0

Highly accurate reference clocks such as atomic clocks and GPS clocks.

Stratum 1

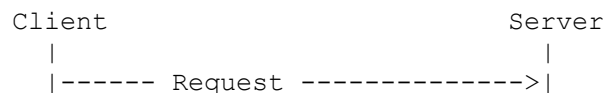
Servers directly synchronized with reference clocks.

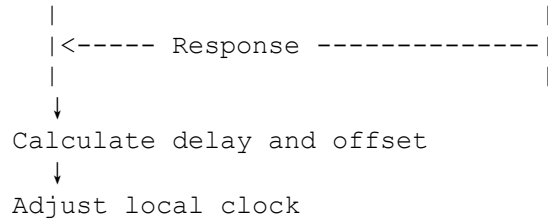
Stratum 2 and higher

Servers synchronize with servers at lower strata.

3. NTP Synchronization Process

NTP exchanges timestamp information between client and server.





4. Important NTP Concepts

Clock Offset

The difference between the local clock and the reference clock.

Round-Trip Delay

The time required for a request and response to travel between client and server.

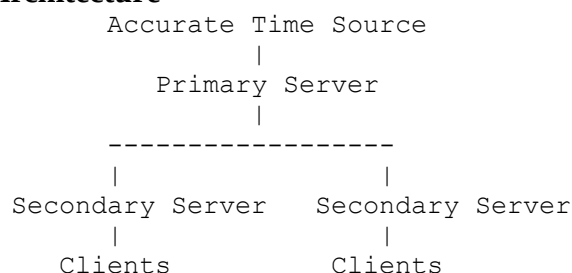
Stratum

Indicates the distance from the reference clock.

5. Advantages of NTP

1. Provides accurate clock synchronization.
2. Supports hierarchical time servers.
3. Works across large networks.
4. Can compensate for network delays.
5. Provides fault tolerance through multiple time sources.

6. NTP Architecture



Conclusion

NTP is a scalable and widely used mechanism for synchronizing physical clocks in distributed systems. It uses a hierarchical arrangement of time sources and estimates clock offset and network delay to maintain accurate system time.

15(a). Explain the Essential Characteristics in Distributed Computing Systems and Their Relation with Computer System Components. (Unit - I)

Answer

A distributed computing system consists of multiple autonomous computers connected through a communication network that cooperate to perform tasks and provide services.

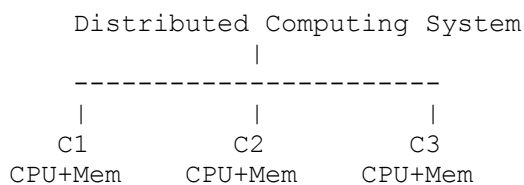
The essential characteristics can be explained as follows.

1. Multiple Autonomous Computers

A distributed system contains several independent computers.

Each computer has its own:

- Processor
- Memory
- Operating system
- Local resources

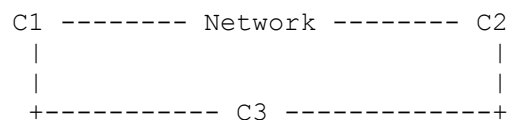


2. Communication Network

The computers communicate using a network.

Examples:

- LAN
- WAN
- Internet
- Wireless networks



3. Resource Sharing

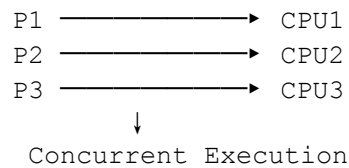
Distributed systems allow users and applications to share:

- Files
- Databases
- Printers

- Storage
- Processing power
- Services

4. Concurrency

Multiple processes can execute concurrently on different computers.



5. No Global Clock

There is no single physical clock shared by all computers.

Therefore, distributed systems use:

- Physical clock synchronization
- Logical clocks
- Vector clocks

6. Independent Failures

One component can fail while other components continue operating.

This makes **fault tolerance** an important characteristic.

7. Transparency

The distributed system should hide unnecessary details of distribution from users.

Important forms include:

- Access transparency
- Location transparency
- Migration transparency
- Replication transparency
- Concurrency transparency
- Failure transparency

8. Scalability

The system should support growth in:

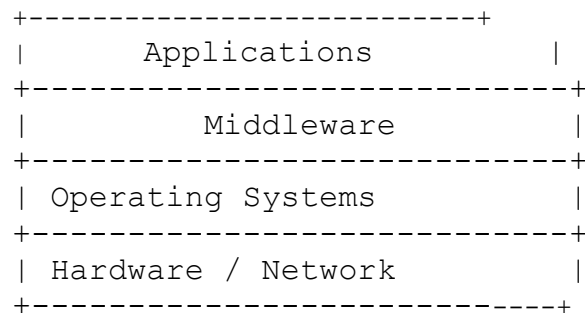
- Number of users
- Number of computers
- Geographic coverage
- Data volume

9. Heterogeneity

Different hardware, operating systems, networks and programming languages can participate in the same distributed system.

10. Middleware

Middleware provides a software layer between applications and the underlying operating systems/network.



Relation with Computer System Components

Component	Role
Processor	Executes distributed processes
Memory	Stores local process and data state
Network	Provides communication
Operating System	Manages local resources
Middleware	Hides distribution and provides common services
Storage	Provides persistent distributed data
Applications	Use distributed services

Conclusion

Distributed computing combines processors, memory, communication networks, operating systems, middleware and applications to provide resource sharing, concurrency, scalability and reliable services.

15(b). Explain the Applications and Challenges in Distributed Computing. (Unit - I)

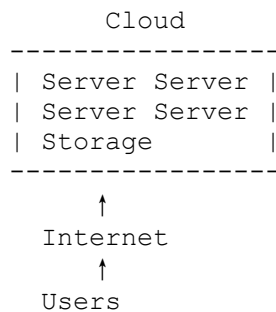
Answer

Distributed computing is used when multiple computers cooperate to perform computation or provide services.

Applications

1. Cloud Computing

Cloud platforms distribute computation and storage across large numbers of servers.



2. Distributed Databases

Data is stored across multiple database servers.

Advantages include:

- Data availability
- Scalability
- Faster access
- Fault tolerance

3. Distributed File Systems

Files are stored and accessed across multiple computers.

4. Web Services

Web applications use distributed servers to provide services to users.

5. Banking Systems

Banks use distributed systems for:

- ATM services
- Online banking
- Transaction processing
- Account management

6. E-Commerce

Online shopping platforms distribute:

- Product databases
- Payment services
- Order processing
- Customer services

7. Scientific Computing

Large computational problems can be divided among multiple computers.

8. Multimedia Systems

Distributed systems support:

- Video streaming
- Audio streaming
- Online conferencing

Challenges

1. Heterogeneity

Different hardware and software technologies must interoperate.

2. Security

Distributed resources must be protected from unauthorized access.

3. Scalability

Performance should not degrade significantly as the system grows.

4. Fault Tolerance

The system must continue operation despite component failures.

5. Synchronization

Processes must coordinate without relying on a common global clock.

6. Communication Delay

Network latency can affect distributed application performance.

7. Concurrency

Simultaneous access to shared resources may cause consistency problems.

8. Resource Management

Resources must be efficiently allocated among distributed processes.

Conclusion

Distributed computing has applications in cloud computing, databases, banking, e-commerce, scientific computing and web services, but challenges such as security, heterogeneity, synchronization, communication delay, scalability and failures must be carefully handled.

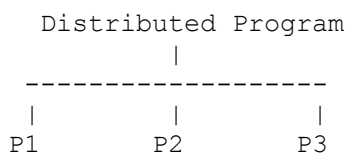
16(a). Explain the Model of Distributed Computations in a Distributed Program. (Unit - I)

Answer

A distributed computation consists of multiple processes executing on different computers and communicating through messages.

1. Processes

A distributed program consists of multiple processes.



Each process has:

- Local state
- Local computation
- Communication operations

2. Events

The execution of a process can be represented as a sequence of events.

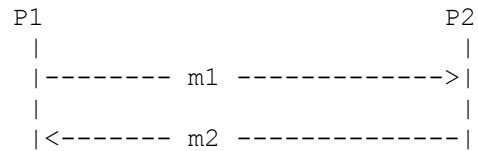
The main events are:

1. Internal computation
2. Message send
3. Message receive

P1: Internal → Send → Internal → Receive
 P2: Internal → Receive → Internal → Send

3. Message Communication

Processes communicate through messages.



4. Local State

The state of each process consists of:

- Local variables
- Program counter
- Local data structures
- Local clock

5. Communication Channels

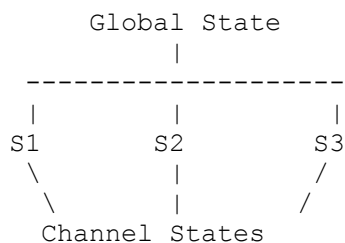
Messages transmitted between processes are carried through communication channels.

A channel can be modeled as a sequence of messages in transit.

6. Global State

The global state consists of:

Global State = Local states of all processes + States of communication channels



7. Event Ordering

Events are related by the happened-before / causal precedence relation.

For example:

P1: a → send(m)

↓

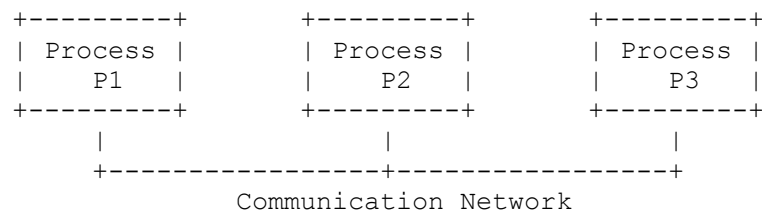
P2: receive(m) → b

Thus:

send(m) → receive(m)

8. Distributed Computation Model

A simplified model is:



Conclusion

The distributed computation model represents a distributed program as a collection of processes that perform local computation and communicate through messages. Events, local states, channels and causal relationships are essential elements of the model.

16(b). Explain the Difference Between Synchronous and Asynchronous Executions.

Answer

Synchronous and asynchronous executions represent two different assumptions about process execution and communication in distributed systems.

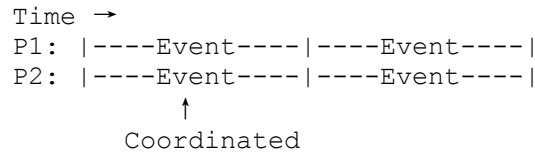
Synchronous Execution

In a synchronous system, assumptions can be made about:

- Process execution time
- Message transmission time
- Clock behavior

Characteristics

1. Processes operate under known timing constraints.
2. Message transmission has a known upper bound.
3. Process execution time has a known bound.
4. Timeouts can be used more reliably.



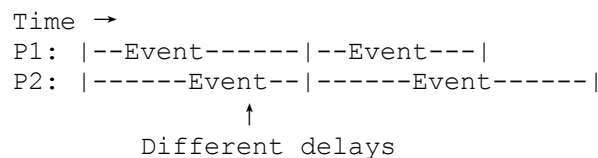
Asynchronous Execution

In an asynchronous system, there are no known upper bounds on:

- Process execution time
- Message transmission delay
- Clock rate differences

Characteristics

1. Processes can execute at different speeds.
2. Messages may experience arbitrary delays.
3. There is no reliable global timing assumption.
4. Failure detection becomes difficult.



Comparison

Synchronous	Asynchronous
Timing bounds are known	Timing bounds are unknown
Message delay is bounded	Message delay may be arbitrary
Process speed is bounded	Process speed may vary
Easier to use timeouts	Timeouts are difficult to interpret
Easier failure detection	Failure detection is difficult

Conclusion

Synchronous execution provides timing guarantees, whereas asynchronous execution makes fewer assumptions about timing and therefore more closely models many real distributed environments.

17(a). Describe Blocking and Non-Blocking Primitives and Synchronous and Asynchronous Primitives in Distributed Communication.(Unit-I)

Answer

Distributed processes communicate using communication primitives. These primitives can be classified based on whether the operation blocks and whether communication requires synchronization.

1. Blocking Primitive

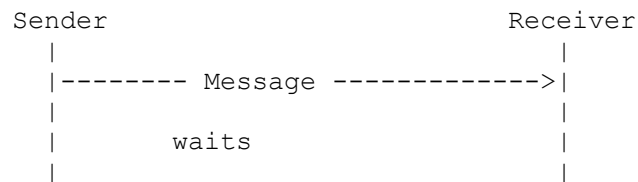
A blocking operation causes the process to wait until the required communication condition is satisfied.

Blocking Send

The sender waits until the required send operation is completed.

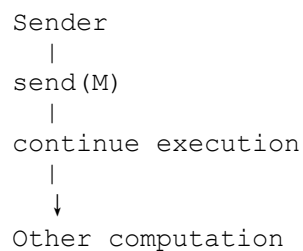
Blocking Receive

The receiver waits until a message becomes available.



2. Non-Blocking Primitive

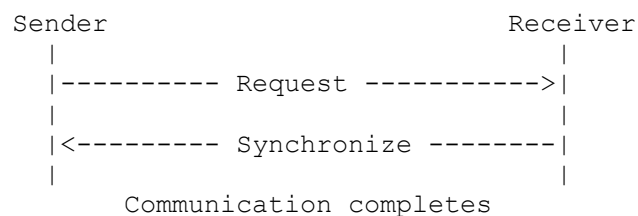
A non-blocking operation allows the process to continue execution without waiting for communication to complete.



3. Synchronous Primitive

Communication involves synchronization between sender and receiver.

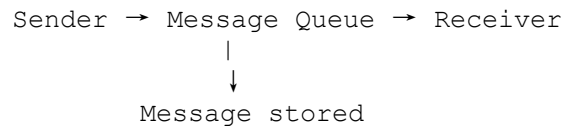
A sender and receiver must reach a communication point.



4. Asynchronous Primitive

The sender can send a message without waiting for the receiver to be ready.

A buffer or queue can store the message.



Comparison

Blocking	Non-blocking
Process waits	Process continues
Easier synchronization	More concurrency
May reduce parallelism	Improves overlap of computation and communication
Synchronous	Asynchronous
Requires communication synchronization	Does not require immediate synchronization
Sender/receiver coordinate	Sender and receiver can operate independently
Often uses rendezvous	Often uses buffering

Conclusion

Blocking and non-blocking primitives determine whether a process waits, while synchronous and asynchronous primitives determine whether communication requires immediate synchronization.

17(b). Explain the Motivation and Need of Distributed Systems.(Unit-I)

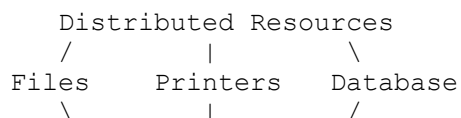
Answer

Distributed systems are developed to allow multiple computers to cooperate and share resources.

1. Resource Sharing

Distributed systems enable sharing of:

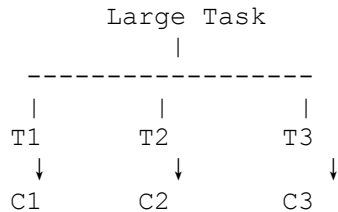
- Hardware
- Software
- Data
- Services
- Storage



Network

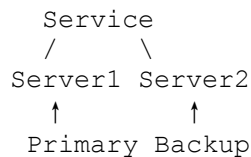
2. Performance

A large computation can be divided among multiple computers.



3. Reliability

Replication allows services to continue when one component fails.



4. Scalability

More computers can be added as demand increases.

5. Geographical Distribution

Resources can be located in different geographical regions and accessed through networks.

6. Communication

Distributed systems support communication between users and applications located at different sites.

7. Cost Effectiveness

A collection of commodity computers can sometimes provide significant computing capacity at lower cost than a single specialized machine.

8. Availability

Replicated services and resources can remain available even when some components fail.

9. Parallel Processing

Independent tasks can be executed concurrently on different processors.

10. Load Sharing

Workloads can be distributed among available computers to improve utilization.

Conclusion

The major motivations for distributed systems are resource sharing, performance, reliability, scalability, availability, communication, geographical distribution and cost effectiveness.

18(a). Explain the Purpose of Message Ordering Paradigms and Provide an Example for Asynchronous Execution Communication. (Unit - II)

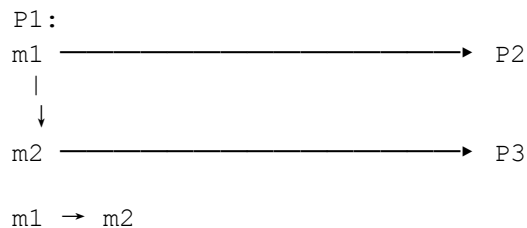
Answer

In distributed systems, messages may reach different processes in different orders because of network delays. Message ordering paradigms provide rules for determining the order in which messages are delivered.

Purpose of Message Ordering

1. Maintaining Causal Relationships

If one message depends on another, the dependent message should not be delivered first.



2. Consistency

Ordering helps distributed processes maintain consistent views of data.

3. Coordination

Processes can coordinate their activities based on an agreed ordering.

4. Replicated Systems

Ordering is important when multiple replicas must process updates consistently.

Message Ordering Paradigms

FIFO Ordering

Messages from the same sender are delivered in their sending order.

P1: m1 → m2 → m3

P2 must receive:

m1 → m2 → m3

Causal Ordering

Causally related messages are delivered in causal order.

m1 → m2

Therefore:

deliver(m1) before deliver(m2)

Total Ordering

All processes deliver messages in the same order.

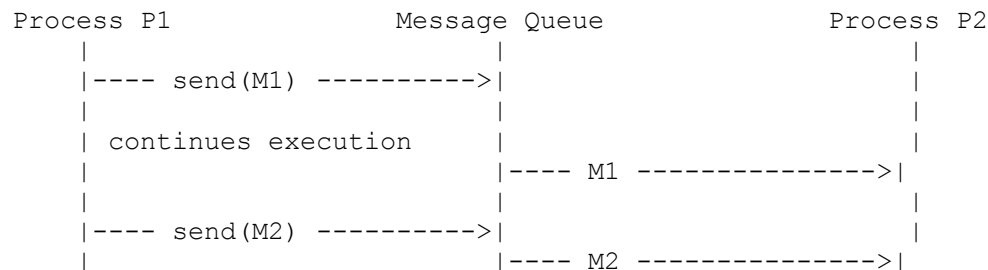
P1: m1 → m2 → m3

P2: m1 → m2 → m3

P3: m1 → m2 → m3

Example of Asynchronous Execution Communication

Consider two processes communicating through a message queue.



Here, P1 does not wait for P2 to receive the message. The queue temporarily stores messages.

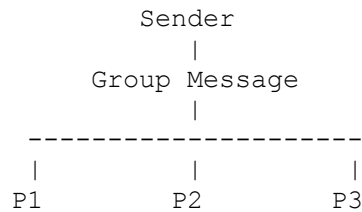
Conclusion

Message ordering is essential for maintaining causal relationships, consistency and coordination. Asynchronous communication allows processes to execute independently while messages are transferred through buffers or queues.

18(b). Explain the Types of Group Communication Used in Distributed Systems. (Unit - II)

Answer

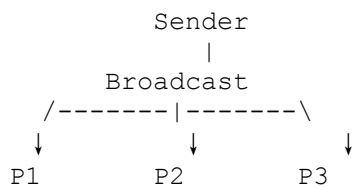
Group communication allows a process to communicate with a collection of processes called a group.



The important forms of group communication are as follows.

1. Broadcast

A message is sent to all processes in the communication network or relevant domain.

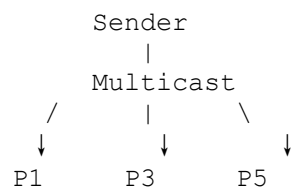


Example

A server sends an update to all connected clients.

2. Multicast

A message is sent to a selected group of processes.



Only members of the specified group receive the message.

3. Unicast

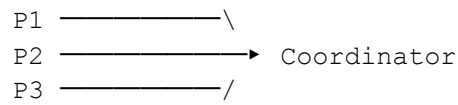
A message is sent to a single process.



Although it is point-to-point rather than group delivery, it is an important basic communication form.

4. Many-to-One Communication

Multiple processes send messages to one process.

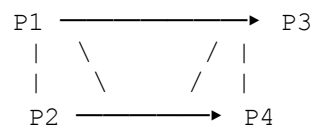


This is useful for:

- Aggregation
- Collection of results
- Coordinator-based algorithms

5. Many-to-Many Communication

Multiple senders communicate with multiple receivers.



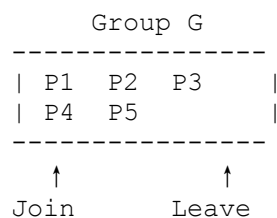
It is useful in collaborative and distributed applications.

6. Group Membership

Group communication also requires management of group membership.

Processes can:

- Join a group.
- Leave a group.
- Be added or removed because of failures.



7. Reliable Group Communication

Messages should be delivered reliably to group members despite communication failures.

8. Ordered Group Communication

Messages can be delivered using:

- FIFO ordering
- Causal ordering
- Total ordering

Conclusion

The major group communication mechanisms include broadcast, multicast, unicast, many-to-one and many-to-many communication. Reliable delivery, group membership and message ordering are important for implementing dependable distributed applications.

19. Explain the characteristics, advantages and limitations of distributed systems with suitable examples. (Unit - I)

BTL: K3

CO: CO1

PI: 1.4.1

Answer

1. Resource Sharing

- Hardware, software, data and services can be shared.
- Improves utilization of available resources.

2. Concurrency

- Multiple processes execute simultaneously.
- Improves system performance.

3. No Global Clock

- Each computer has its own clock.
- Logical and physical clock synchronization mechanisms are required.

4. Independent Failures

- Individual computers or communication links may fail.
- Other components should continue operation where possible.

5. Scalability

- New computers and resources can be added.
- The system should continue to perform efficiently.

6. Heterogeneity

- Different hardware, operating systems and networks can participate.

7. Transparency

- Distribution should be hidden from users.
- Includes location, access, replication and failure transparency.

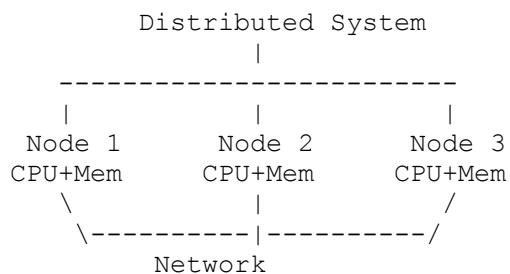
Advantages

1. Resource sharing.
2. Improved performance.
3. Increased reliability.
4. Scalability.
5. Geographical distribution.

Limitations

1. Network failures.
2. Security issues.
3. Synchronization problems.
4. Complex resource management.
5. Communication delays.

Diagram



20. Explain the major applications of distributed systems in real-world computing environments.

BTL: K3

CO: CO1

PI: 1.3.1

Answer

Distributed systems are extensively used in modern computing.

1. Cloud Computing

- Computing resources are distributed across data centres.
- Provides on-demand processing and storage.

2. Distributed Databases

- Data is distributed across multiple database servers.
- Improves availability and scalability.

3. Banking Systems

- ATMs, branches and online banking systems communicate through distributed networks.

4. E-Commerce

- Web servers, databases, payment services and order-processing systems work together.

5. Distributed File Systems

- Files are stored across multiple machines.
- Users can access files as if they were locally stored.

6. Social Networking

- User data, services and media are distributed across many servers.

7. Scientific Computing

- Large computational problems are divided among multiple computers.

8. Multimedia Systems

- Supports video streaming, online meetings and multimedia delivery.

9. Internet Services

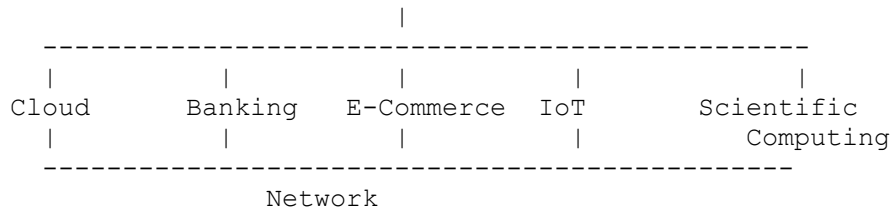
- DNS, web services and content delivery systems use distributed architectures.

10. IoT Systems

- Sensors, gateways, edge devices and cloud platforms cooperate to process data.

Diagram

Distributed Applications



21. Analyze the issues of transparency, flexibility and reliability in distributed systems. (Unit-I)

BTL: K4

CO: CO1

PI: 2.1.1

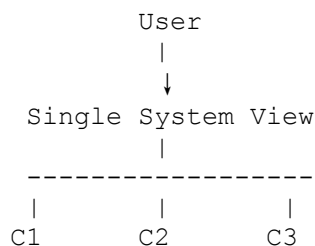
Answer

1. Transparency

Transparency hides the distributed nature of the system.

Types

- Access transparency
- Location transparency
- Migration transparency
- Replication transparency
- Concurrency transparency
- Failure transparency



2. Flexibility

A distributed system should be adaptable to changes.

It should support:

- Addition of new resources.
- Replacement of components.
- Different operating systems.
- Different communication mechanisms.

3. Reliability

Reliability means the system should continue to provide services despite failures.

Methods include:

- Replication.
- Redundancy.
- Checkpointing.
- Recovery.
- Fault detection.

Relationship

Transparency
+
Flexibility
+
Reliability
↓
Effective Distributed System

Conclusion

Transparency improves usability, flexibility improves adaptability, and reliability ensures continued service availability.

22. Explain the role of communication networks, operating systems, middleware and applications in a distributed computing environment. (Unit - I)

BTL: K3

CO: CO1

PI: 1.4.1

Answer:

A distributed computing environment consists of several interacting layers.

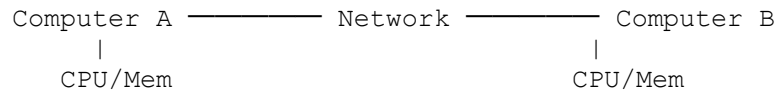
1. Hardware

Includes:

- CPUs
- Memory
- Storage
- Network interfaces

2. Communication Network

Provides communication between computers.



3. Operating System

Manages:

- Processes
- Memory
- Files
- Local resources
- Communication

4. Middleware

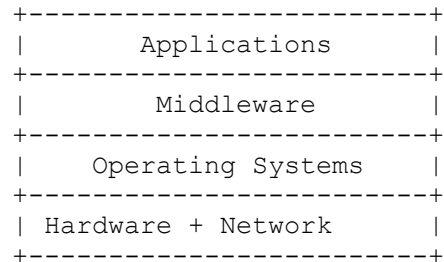
Provides common distributed services.

Examples:

- Remote procedure call
- Naming
- Message-oriented middleware
- Distributed object services

5. Applications

Applications use distributed services to provide functionality to users.



Conclusion

These layers work together to provide transparent, scalable and reliable distributed computing services.

**23. Explain the challenges involved in designing scalable and secure distributed systems.
(Unit-II)**

BTL: K4

CO: CO1

PI: 2.1.1

Answer:

1. Scalability Challenges

a. Size

The number of nodes and users may increase significantly.

b. Geographic Distribution

Nodes may be located in different geographical regions.

c. Administrative Scalability

Different organizations may control different components.

d. Communication Overhead

Increasing the number of nodes increases communication.

e. Resource Management

Resources must be allocated efficiently.

2. Security Challenges

a. Authentication

Verifies the identity of users and systems.

b. Authorization

Controls access to resources.

c. Confidentiality

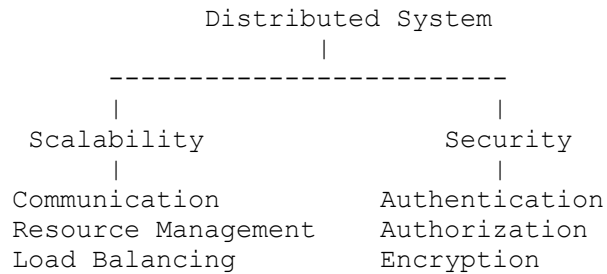
Protects information from unauthorized access.

d. Integrity

Protects data from unauthorized modification.

e. Availability

Ensures services remain accessible.



Conclusion

A scalable distributed system must handle growth efficiently, while a secure system must protect data, communication and resources.

24. Explain Lamport's logical clock algorithm with rules and a suitable example. (Unit-II)

BTL: K3

CO: CO2

PI: 2.1.3

Answer:

Lamport logical clocks are used to establish a logical ordering of events in distributed systems.

Rules

Rule 1 – Before Every Local Event

Increment the logical clock.

$$C = C + 1$$

Rule 2 – Sending a Message

The sender increments its clock and attaches the timestamp to the message.

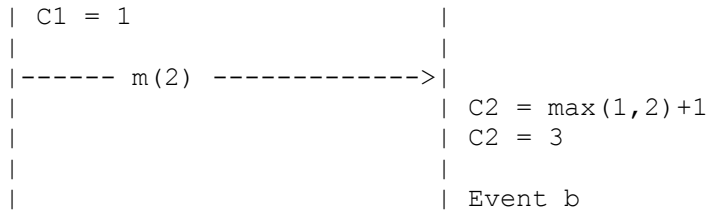
Rule 3 – Receiving a Message

If the received timestamp is T , the receiver updates:

$$C = \max(C, T) + 1$$

Example





Advantages

1. Simple to implement.
2. Provides ordering of events.
3. Does not require synchronized physical clocks.
4. Useful in distributed algorithms.

Limitation

Lamport clocks cannot determine all concurrency relationships.

25. Explain vector clocks and show how they are used to identify causal relationships among events. (Unit-II)

BTL: K4

CO: CO2

PI: 2.1.3

Answer:

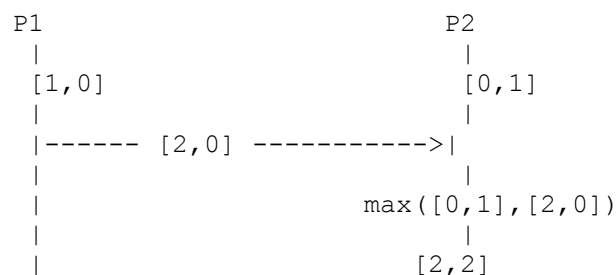
A vector clock assigns a vector to every event in a distributed system.

For n processes, each vector contains n entries.

Rules

1. Each process increments its own vector entry for a local event.
2. A process sends its vector along with a message.
3. The receiver takes the component-wise maximum.
4. The receiver increments its own component.

Example



Causal Relationship

If:

$$V(a) < V(b)$$

component-wise, then:

$$a \rightarrow b$$

If neither vector is less than the other, the events may be concurrent.

Advantages

1. Captures causal relationships.
2. Detects concurrent events.
3. More informative than scalar clocks.
4. Useful in distributed databases and replicated systems.

26. Explain physical clock synchronization using Cristian's algorithm and Berkeley algorithm. (Unit-II)

BTL: K3

CO: CO2

PI: 1.1.2

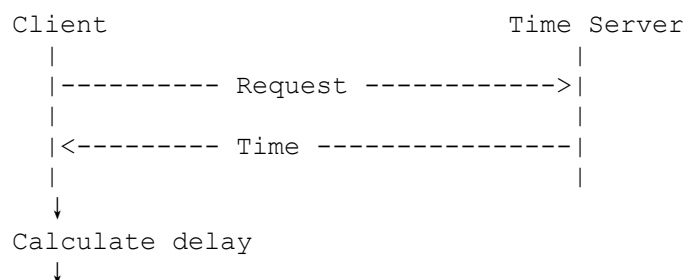
Answer Key

Physical clocks on different machines may differ because of clock drift and clock skew.

Cristian's Algorithm

Steps

1. Client sends a request to the time server.
2. Server returns its current time.
3. Client estimates the communication delay.
4. Client adjusts its clock.

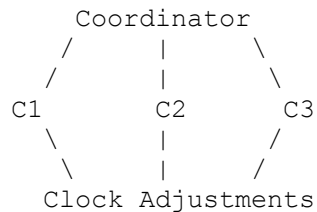


Adjust clock

Berkeley Algorithm

Steps

1. A coordinator polls all machines.
2. Each machine reports its clock value.
3. Coordinator calculates an average time.
4. Corrections are sent to participating machines.



Comparison

Cristian's

Uses a time server

Client synchronizes with server

Server provides reference time

Berkeley

Uses a coordinator

Group synchronizes around an average

Coordinator calculates adjustments

27. Analyze the different message ordering mechanisms used in distributed systems with examples. (Unit-II)

BTL: K4

CO: CO2

PI: 2.2.3

Answer Key

Message ordering ensures that distributed processes deliver messages according to specified ordering rules.

1. FIFO Ordering

Messages from the same sender are delivered in sending order.

P1: m1 → m2 → m3

P2: m1 → m2 → m3

2. Causal Ordering

If:

$m1 \rightarrow m2$

then $m1$ must be delivered before $m2$.

3. Total Ordering

All processes deliver messages in exactly the same order.

P1: $m1 \rightarrow m2 \rightarrow m3$

P2: $m1 \rightarrow m2 \rightarrow m3$

P3: $m1 \rightarrow m2 \rightarrow m3$

4. Causal-Total Ordering

Combines causal ordering with total ordering.

Importance

1. Maintains consistency.
2. Preserves causal relationships.
3. Supports replicated services.
4. Helps coordinate distributed processes.

Conclusion

The choice of ordering mechanism depends on the consistency and coordination requirements of the distributed application.

28. Explain group communication and discuss multicast, broadcast, reliable delivery and ordered delivery. (Unit-II)

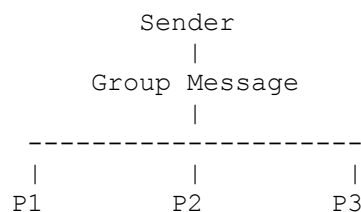
BTL: K3

CO: CO2

PI: 2.2.1

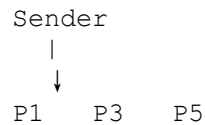
Answer:

Group communication allows processes to communicate with a collection of processes called a group.



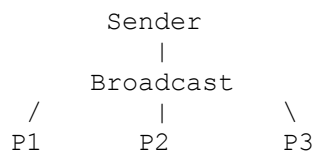
1. Multicast

A message is sent to a selected group.



2. Broadcast

A message is sent to all relevant processes.



3. Reliable Group Communication

A message should be delivered to all intended group members despite communication problems.

Important features:

- Message retransmission.
- Failure detection.
- Delivery guarantees.
- Membership management.

4. Ordered Group Communication

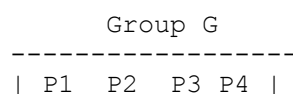
Messages can be delivered according to:

- FIFO order.
- Causal order.
- Total order.

5. Group Membership

Processes can:

- Join a group.
- Leave a group.
- Be removed after failure.





Applications

1. Replicated servers.
2. Distributed databases.
3. Collaborative applications.
4. Fault-tolerant services.
5. Multimedia and conferencing systems.

Conclusion

Group communication provides an efficient mechanism for coordinating multiple distributed processes. Multicast, broadcast, reliable delivery and ordered delivery are fundamental mechanisms for building dependable distributed applications.